



FOD

Maestro Detalle

Archivo maestro: resume información sobre el dominio de un problema específico. Por ejemplo, el archivo de productos de una empresa.

Archivo detalle: contiene movimientos realizados sobre la información almacenada en el maestro. Por ejemplo, el archivo contiene las ventas realizadas sobre esos productos.

Importante: Analizar las precondiciones de cada caso particular. Los algoritmos a desarrollar deben tener en cuenta estas precondiciones, caso contrario determina la falla de su ejecución.

Actualización de un archivo maestro con un archivo detalle - Precondiciones

- Existe un archivo maestro.
- Existe un único archivo detalle que modifica al maestro.
- Cada registro del detalle modifica a un solo registro del maestro que seguro existe.

- No todos los registros del maestro son necesariamente modificados.
- Cada elemento del maestro que se modifica, es alterado por un solo elemento del archivo detalle.
- Ambos archivos están ordenados por igual criterio.

```
{Ejemplo: Definición de tipos}
type
    producto = record
        cod: string[4];
        descripcion: string[30];
        pu: real; {precio unitario}
        stock: integer;
    end;
    venta_prod = record
        cod: string[4];
        cant_vendida: integer;
    end;
    maestro = file of producto;
    detalle = file of venta_prod;
```

```
{Ejemplo: variables y operaciones}
var
    mae: maestro;
    det: detalle;
    regm: producto;
    regd: venta_prod;
begin { Inicio del programa }
    assign(mae, 'maestro.dat');
    assign(det, 'detalle.dat');
    reset(mae);
    reset(det);
```

```

{Loop archivo detalle}
while not(EOF(det)) do begin
    read(mae, regm); // Lectura archivo maestro
    read(det, regd); // Lectura archivo detalle
    {Se busca en el maestro el producto del
    detalle}
    while (regm.cod <> regd.cod) do
        read(mae, regm);
    {Se modifica el stock del producto con la
    cantidad vendida de ese producto}
    regm.stock := regm.stock-regd.cant_vendida;
    {Se reubica el puntero en el maestro}
    seek(mae, filepos(mae)-1);
    {Se actualiza el maestro}
    write(mae, regm);
end; // Fin while archivo detalle
close(det);
close(mae);
end.

```

Actualización de un archivo maestro con un archivo detalle

- Existe un archivo maestro.
- Existe un único archivo detalle que modifica al maestro.
- Cada registro del detalle modifica a un registro del maestro que seguro existe.
- No todos los registros del maestro son necesariamente modificados.
- Cada elemento del archivo maestro puede o no ser modificado, o ser **modificado por uno o mas elementos del detalle**.
- Ambos archivos están ordenados por el mismo criterio.

```

{Ejemplo: variables y operaciones}
var
    mae: maestro;

```

```

    det: detalle;
    regm: producto;
    regd: venta_prod;
    cod_actual: string[4];
    tot_vendido: integer;
begin { Inicio del programa }
    assign(mae, 'maestro.dat');
    assign(det, 'detalle.dat');
    reset(mae);
    reset(det);

{Loop archivo detalle}
while not(EOF(det)) do begin
    read(mae, regm); // Lectura archivo maestro
    read(det, regd); // Lectura archivo detalle
    {Se busca en el maestro el producto del
    detalle}
    while (regm.cod <> regd.cod) do
        read(mae, regm);
    {Se totaliza la cantidad vendida del detalle}
    cod_actual := regd.cod;
    tot_vendido := 0;
    while (regd.cod = cod_actual) do begin
        tot_vendido:=tot_vendido+regd.cant_vendida;
        read(det, regd);
    end;
    {Se actualiza el stock del producto con la
    cantidad vendida del mismo}
    regm.stock := regm.stock - tot_vendido;
    {se reubica el puntero en el maestro}
    seek(mae, filepos(mae)-1);
    {se actualiza el maestro}
    write(mae, regm);
end;
close(det);

```

```
close(mae);  
end.
```

¿Diferencia entre este ejemplo y el anterior?

Se agrega una iteración que permite agrupar todos los registros del detalle que modificaran a un elemento del maestro.

¿Inconvenientes de esta solución?

La segunda operación read sobre el archivo detalle se hace sin controlar el fin de datos del mismo. Podría solucionarse agregando un if que permita controlar dicha operación, pero cuando finaliza la iteración interna, al retornar a la iteración principal se lee otro registro del archivo detalle, perdiendo así un registro.

Corte de control

Proceso mediante el cual la información de un archivo es presentada en forma organizada de acuerdo a la estructura que posee el archivo.

Ejemplo: se almacena en un archivo la información de ventas de una cadena de electrodomésticos. Dichas ventas han sido efectuadas por los vendedores de cada sucursal de cada ciudad de cada provincia del país.

Es necesario informar al gerente de ventas de la empresa el total vendido en cada sucursal, ciudad y provincia, así como el total final.

Precondiciones:

- El archivo se encuentra ordenado por provincia, ciudad y sucursal.
- En diferentes provincias pueden existir ciudades con el mismo nombre, y en diferentes ciudades pueden existir sucursales de igual denominación.

```
program ejemplo;  
const VALOR_ALTO = 'ZZZ';  
type  
    nombre = string[30];  
    reg_venta = record
```

```

        vendedor: integer;
        monto: real;
        sucursal: nombre;
        ciudad: nombre;
        provincia: nombre;
    end;
    ventas = file of reg_venta;

var
    reg: reg_venta;
    archivo: ventas;
    total, totProv, totCiudad, totSuc: real;
    prov, ciudad, sucursal: nombre;

procedure leer(var archivo: ventas; var dato: reg_venta);
begin
    if (not(EOF(archivo))) then
        read(archivo,dato)
    else
        dato.provincia := VALOR_ALT0;
    end;

{PROGRAMA PRINCIPAL}
begin
    assign(archivo, 'archivo_ventas');
    reset(archivo);

    leer(archivo,reg);
    total := 0;

    while (reg.provincia <> VALOR_ALT0) do begin
        writeln('Provincia:', reg.provincia);
        prov := reg.provincia;
        totProv := 0;
        while (prov = reg.provincia) and (ciudad = reg.ciudad) do begin

```

```

        writeln('Sucursal:', reg.sucursal);
        sucursal = reg.sucursal;
        totSuc := 0;
        while (prov = reg.provincia) and (ciudad = reg.ciudad) do begin
            write("Vendedor:", reg.vendedor);
            writeln(reg.monto);
            totSuc := totSuc + reg.monto;
            leer(archivo, reg);
        end;
        writeln('Total sucursal', totSuc);
        totCiudad := totCiudad + totSuc;
    end;
    writeln("Total Ciudad", totCiudad);
    totProv := totProv + totCiudad;
end;
writeln("Total Empresa", total);
close(archivo);
end.

```

Merge

Es una técnica que sirve para combinar dos o mas archivos (o secuencias de datos) que ya están ordenados, generando un nuevo archivo también ordenado.

- Se mira el primer registro de cada archivo.
- Se compara según la clave de orden (por ejemplo DNI, código, apellido, etc).
- Se copia al archivo resultado el menor (o mayor, según criterio).
- Se avanza solo en el archivo del que salió ese dato.
- Se repite hasta terminar todos los archivos.

Este programa genera un archivo maestro a partir de 3 archivos detalle ordenados por código de producto, escribiendo los registros en orden creciente según dicho código mediante un proceso de Merge (intercalación).

```

program ejemplo;
const VALOR_ALTO = 999999;
type
    producto = record
        codigo: LongInt;
        descripcion: string[30];
        pu: real;
        cant: integer;
    end;
    arc_productos = file of producto;
var
    det1, det2, det3, mae: arc_productos;
    min, regd1, regd2, regd3: producto;

procedure leer(var archivo: arc_productos; var dato: producto);
begin
    if (not(EOF(archivo))) then
        read (archivo, dato)
    else
        dato.codigo = VALOR_ALTO;
end;

procedure minimo(var det1, det2, det3: arc_productos; var r1,
r2, r3, min: producto);
begin
    if (r1.codigo <= r2.codigo) and (r1.codigo <= r3.codigo)
then begin
        min := r1;
        leer(det1, r1);
    end
    else
        if (r2.cod <= r3.cod) then begin
            min = r2;
            leer(det2, r2);
        end
    end
end;

```

```

        end
    else begin
        min := r3;
        leer(det3, r3);
    end;
end;

{PROGRAMA PRINCIPAL}
begin
    assign (mae, 'maestro');
    assign (det1, 'detalle1');
    assign (det2, 'detalle2');
    assign (det3, 'detalle3');
    rewrite (mae);
    reset (det1);
    reset (det2);
    reset (det3);
    leer (det1, regd1);
    leer (det2, regd2);
    leer (det3, regd3);
    minimo(det1, det2, det3, regd1, regd2, regd3, min);
    {se procesan todos los registros de los archivos detalle}
    while (min.codigo <> VALOR_ALT0) do begin
        write(mae,min);
        minimo(det1, det2, det3, regd1, regd2, regd3, min);
    end;
    close(det1);
    close(det2);
    close(det3);
    close(mae);
end.

```

Archivos - Bajas

Se denomina proceso de baja a aquel proceso que permite quitar información de un archivo. El proceso de baja puede llevarse a cabo de dos modos diferentes:

Baja Física: consiste en borrar efectivamente la información del archivo, recuperando el espacio físico.

Se realiza una baja física sobre un archivo cuando un elemento es efectivamente quitado del archivo, decrementando en uno la cantidad de elementos.

VENTAJA: en todo momento, se administra un archivo de datos que ocupa el lugar mínimo necesario.

DESVENTAJA: performance de los algoritmos que implementan esta solución.

Baja Lógica: consiste en borrar la información del archivo, pero sin recuperar el espacio físico respectivo.

Técnicas de Baja Física

- Generar un nuevo archivo con los elementos válidos (sin copiar los que se desea eliminar).
- Utilizar el mismo archivo de datos, generando los reacomodamientos que sean necesarios. (Solo para archivos sin ordenar).

```
program ejemplo;

begin
  {Se sabe que existe Carlos Garcia}
  assign(archivo, 'arch_empleados');
  assign (archivo_nuevo, 'arch_nuevo');
  reset (archivo);
  rewrite (archivo_nuevo);
  leer (archivo, reg);
  {Se copian los registros previos a Carlos Garcia}
  while (reg.nombre <> 'Carlos Garcia') do
    begin
      write(archivo_nuevo, reg);
      leer(archivo,reg);
    end
  end
```

```

end;
{Se descarta a Carlos Garcia}
leer(archivo,reg);
{Se copian los registros restantes}
while (reg.nombre <> valoralto) do begin
write(archivo_nuevo, reg);
leer(archivo, reg);
end;
close(archiv_nuevo);
close(archivo);
{renombrar el archivo original para dejarlo como respaldo}
rename(archivo,'arch_empleados_old');
{renombrar el archivo temporal con el nombre del original}
rename(archivo_nuevo, 'arch_empleados');
end.

```

Ejemplo de baja lógica.

```

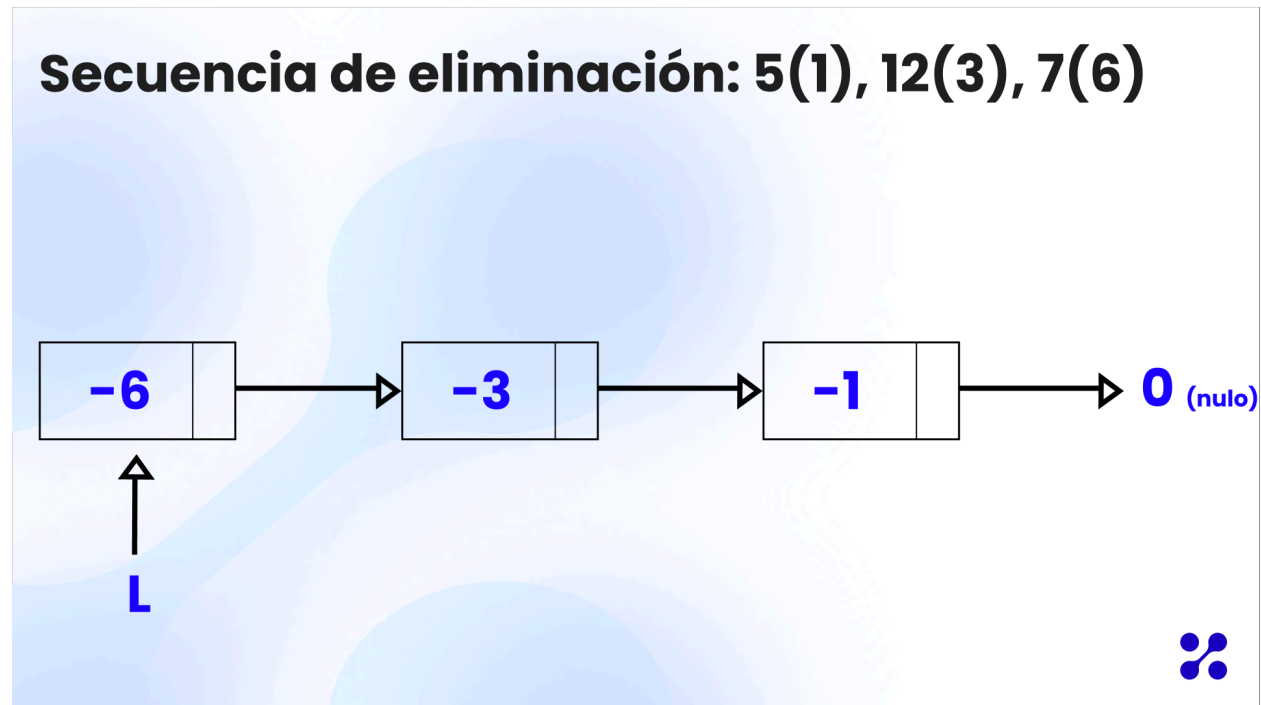
program ejemplo;

Begin {se sabe que existe Carlos Garcia}
    assign(archivo, 'arch_empleados');
    reset(archivo);
    leer(archivo, reg);
    {Se avanza hasta Carlos Garcia}
    while (reg.nombre <> 'Carlos Garcia') do
        leer(archivo, reg);
    {Se genera una marca de borrado}
    reg.nombre := '***';
    {Se borra lógicamente a Carlos Garcia}
    seek(archivo, filepos(archivo)-1 );
    write(archivo, reg);
    close(archivo);
end.

```

Lista Invertida

Es una lista enlazada que guarda en cada nodo la -(posición) del ultimo elemento eliminado lógicamente del archivo, con un puntero al siguiente espacio libre. El 'nil' seria 0.



Árboles B

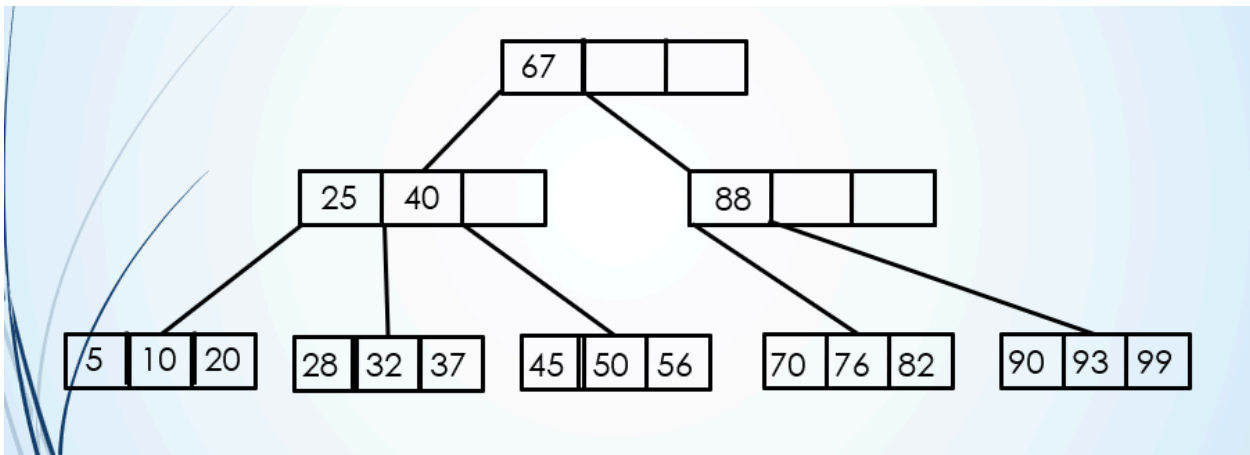
Los arboles B son arboles multicamino con una construcción especial en forma ascendente que permite mantenerlos balanceados a bajo costo.

Propiedades de un Árbol B de orden M

- Cada nodo puede contener como **máximo M descendientes** directos (hijos).
- La raíz no tiene descendientes directos o tiene al menos dos.
- Un nodo interno con X hijos contiene X-1 elementos.
- Todos los nodos (salvo la raíz) tienen como mínimo $\lceil M/2 \rceil - 1$ elementos y como máximo M - 1 elementos.
- Todos los nodos terminales se encuentran al mismo nivel.

- Cada nodo tiene sus elementos ordenados por clave.
Además, todos los elementos en el subárbol izquierdo de un elemento son menores o iguales que dicho elemento, mientras que todos los elementos en el subárbol derecho son mayores.

Ejemplo de Arbol B de orden 4.



Para que usamos los arboles B?

- Organizar el archivo de datos como un árbol B.
- Organizar el archivo índice en un árbol B.

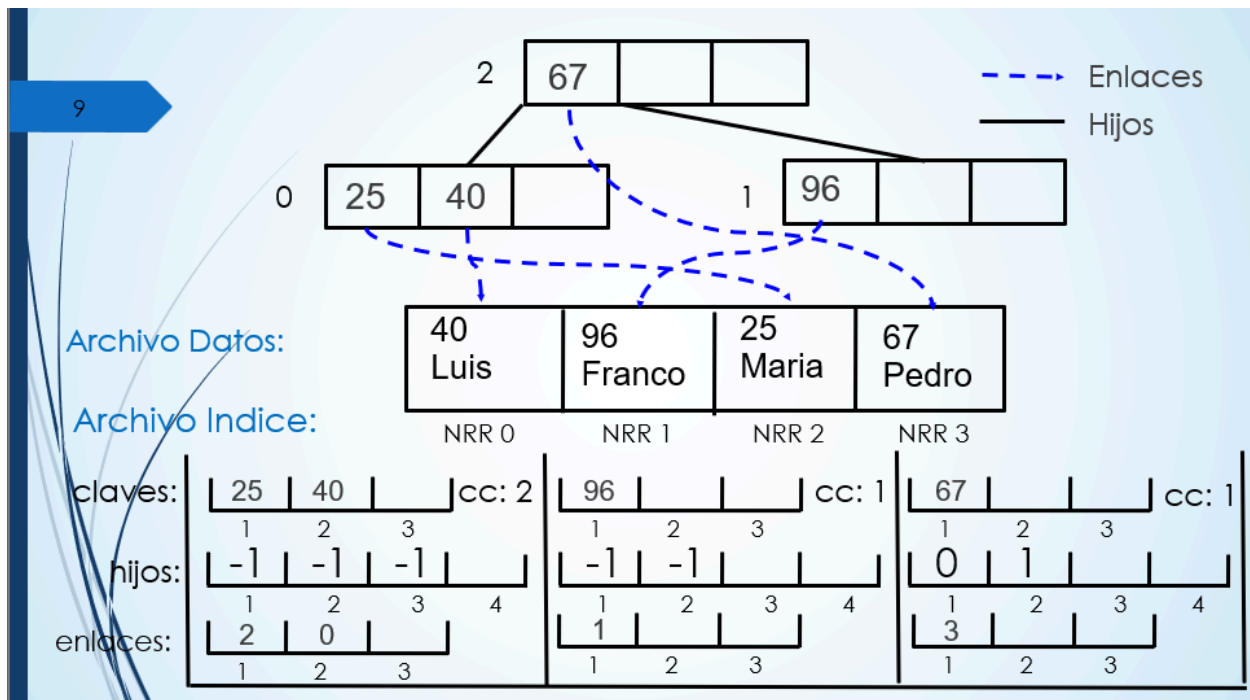
```

const M = ... ; {orden del árbol}
type
  Tdato = record
    codigo: longint;
    nombre: string[50];
  end;
  Tnodo = record
    cant_datos: integer;
    datos: array[1..M-1] of Tdato;
    hijos: array[1..M] of integer;
  end;
  
```

```
    arbolB = file of TNode;  
var    archivoDatos: arbolB;
```

Archivo Indice como Arbol B

```
const M = ... ; {orden del árbol}  
type  
TData = record  
    codigo: longint;  
    nombre: string[50];  
end;  
TNode = record  
    cant_claves:    integer;  
    claves: array[1..M-1] of longint;  
    enlaces: array [1..M-1] of integer;  
    hijos: array[1..M] of integer;  
end;  
TArchivoDatos = file of TData;  
arbolB = file of TNode;  
var    archivoDatos: TArchivoDatos;  
        archivoIndice: arbolB;
```



Overflow

- Se crea un nuevo nodo.
- La primera mitad de las claves se mantiene en el nodo con overflow.
- La segunda mitad de las claves se traslada al nuevo nodo.
- La menor de las claves de la segunda mitad se promociona al nodo padre.

Bajas

- Si la clave a eliminar no esta en una hoja, se debe reemplazar con la menor clave del subárbol derecho.
- Si el nodo hoja contiene por lo menos el mínimo numero de claves, luego de la eliminación, no se requiere ninguna accion adicional.
- En caso contrario se debe tratar el underflow.
- Primero se intenta redistribuir con un hermano adyacente. La redistribución es un proceso mediante el cual se trata de dejar cada nodo lo mas equitativamente cargado posible.
- Si la redistribución no es posible, entonces se debe fusionar con el hermano adyacente.

Políticas para la resolución de underflow

- **Política Izquierda:** se intenta redistribuir con el hermano adyacente izquierdo, si no es posible, se fusiona con hermano adyacente izquierdo.
- **Política derecha:** se intenta redistribuir con el hermano adyacente derecho, si no es posible, se fusiona con hermano adyacente derecho.
- **Política izquierda o derecha:** se intenta redistribuir con el hermano adyacente izquierdo, si no es posible, se intenta con el hermano adyacente derecho, si tampoco es posible, se fusiona con hermano adyacente izquierdo.
- **Política derecha o izquierda:** se intenta redistribuir con el hermano adyacente derecho, si no es posible, se intenta con el hermano adyacente izquierdo, si tampoco es posible, se fusiona con hermano adyacente derecho.
- **Casos especiales:** en cualquier política si se tratase de un nodo hoja de un extremo del árbol debe intentarse redistribuir con el hermano adyacente que el mismo posea.
- **Aclaración:** en caso de underflow lo primero que se intenta **SIEMPRE** es redistribuir si el hermano adyacente se encuentra en condiciones de hacer la redistribución y no se produce underflow en él.

Arboles B+

- Constituyen una mejora sobre los arboles B, pues conservan la propiedad de acceso indizado rápido y permiten además un recorrido secuencial rápido.\
- **Conjunto índice:** proporciona acceso indizado a los registros. Todas las claves se encuentran en las hojas, duplicándose en la raíz y nodos interiores aquellas que resulten necesarias para definir los caminos de búsqueda.
- **Conjunto secuencia:** contiene todos los registros del archivo. Las hojas se vinculan para facilitar el recorrido secuencial rápido. Cuando se lee en orden lógico, lista todos los registros por el orden de la clave.

Búsqueda B+

La operación de búsqueda en árboles B+ es similar a la operación de búsqueda en árboles B. El proceso es simple, ya que todas las claves se encuentran en las

hojas, deberá continuarse con la búsqueda hasta el último nivel del árbol.

Inserción B+

Dificultad: la inserción en nodo lleno (overflow). El nodo afectado se divide en 2, distribuyéndose las claves lo más equitativamente posible. Una copia de la clave del medio o de la menor de las claves mayores (casos de overflow con cantidad pares de elementos) se promociona al nodo padre. El nodo con overflow se divide a la mitad.

La copia de la clave solo se realiza en un overflow ocurrido a nivel hoja.

Caso contrario → igual que en árboles B.

Bajas en B+

La operación de eliminación en árboles B+ es más simple que en árboles B. Esto ocurre porque las claves a eliminar siempre se encuentran en las páginas hojas. En general deben distinguirse los siguientes casos, dado un árbol B+ de orden M:

- Si al eliminar una clave, la cantidad de claves que queda es mayor o igual que $\lceil M/2 \rceil - 1$, entonces termina la operación. Las claves de los nodos raíz o internos no se modifican por más que sean una copia de la clave eliminada en las hojas.

Underflow

- Si al eliminar una clave, la cantidad de llaves es menor a $\lceil M/2 \rceil - 1$, entonces debe realizarse una redistribución de claves, tanto en el índice como en las páginas hojas.
- Si la redistribución no es posible, entonces debe realizarse una fusión entre los nodos.

Hashing

- Técnica para generar una dirección base única para una clave dada.
- Convierte la clave en un número aleatorio, que luego sirve para determinar donde se almacena la clave.

- Utiliza una función de dispersión para mapear cada clave con una dirección física de almacenamiento.
- Utilizada cuando se requiere acceso rápido por clave.

Direccionamiento Dinámico: El espacio disponible para dispersar los registros del archivo aumenta o disminuye en función de las necesidades.

Función de Dispersión: caja negra que a partir de una clave genera la dirección física donde debe almacenarse el registro.

Colisión: situación en la que un registro es asignado, por función de dispersión, a una dirección que ya posee uno o mas registros.

Desborde: situación en la cual una clave carece de lugar en la dirección asignada por la función de dispersión.

Densidad de empaquetamiento: relación entre el espacio disponible para el archivo de datos y la cantidad de registros que integran el mismo.

Hashing Extensible

- Función de dispersión: retorna 10 bits.
- Capacidad para 2 registros por dirección.
- Se van a dispersar 8 claves en total.

Claves a dispersar	
1- Colapinto (1011001100)	2- Verstappen (1110101000)
3- Russell (1010001001)	4- Stroll (1010101010)
5- Alonso (1010001000)	6- Hamilton (1001001011)
7- Sainz (1010001111)	8- Leclerc (1010100111)

Que el numero de bits de dispersión este en 0, indica que no es necesario ningún bit de la secuencia obtenida por la función de dispersión.

Estado inicial del archivo:

Tabla de dispersión Bits de dispersión: 0	
Sufijo	#Bloque
(0)	0

Archivo de datos			
#Bloque	Bits	Clave R1	Clave R2
0	0		

Se agregan Colapinto (011001100) y Verstappen (110101000)
Ambos se agregan sin inconveniente en el bloque 0.

Tabla de dispersión Bits de dispersión: 0	
Sufijo	#Bloque
(0)	0

Archivo de datos			
#Bloque	Bits	Clave R1	Clave R2
0	0	Colapinto (011001100)	Verstappen (110101000)

Se agrega Russel (1010001001) → produce desborde (overflow)

Tabla de dispersión Bits de dispersión: 1		Archivo de datos			
Sufijo	#Bloque	#Bloque	Bits	Clave R1	Clave R2
(0)	1	0	1	Russel (1010001001)	
(1)	0	1	1	Colapinto (011001100)	Verstappen (110101000)

Se produce desborde en el bloque 0. Aumentamos en uno los bits de dispersión locales del bloque 0 y creamos un nuevo bloque (bloque 1) con la misma cantidad de bits locales. Luego comparamos los bits locales del bloque con los bits de dispersión globales de la tabla: como ahora el bloque tiene más bits locales que los bits globales de la tabla, aumentamos en uno los bits globales de la tabla y duplicamos la cantidad de direcciones. Finalmente, las claves se redistribuyen entre el bloque original y el nuevo según el bit menos significativo correspondiente. La dirección donde ocurrió el desborde ahora apunta al nuevo bloque.

Hashing Extensible

Se agrega Alonso (1010001000) → produce desborde

Se agrega Hamilton (1001001011) → entra normal

Se agrega Sainz (1010001111) → produce desborde

Se agrega Leclerc (1010100111) → produce desborde

Tabla de dispersión Bits de dispersión: 3	
Sufijo	#Bloque
(000)	3
(001)	0
(010)	1
(011)	4
(100)	2
(101)	0
(110)	1
(111)	5

Archivo de datos			
#Bloque	Bits	Clave R1	Clave R2
0	2	Russell (1010001001)	
1	2	Stroll (1010101010)	
2	3	Colapinto (1011001100)	
3	3	Alonso (1010001000)	Verstappen (1110101000)
4	3	Hamilton (1001001011)	
5	3	Leclerc (1010100111)	Sainz (1010001111)

Doy de baja a Verstappen (1110101000) → se borra normal.

Tabla de dispersión Bits de dispersión: 3	
Sufijo	#Bloque
(000)	3
(001)	0
(010)	1
(011)	4
(100)	2
(101)	0
(110)	1
(111)	5

Archivo de datos			
#Bloque	Bits	Clave R1	Clave R2
0	2	Russell (1010001001)	
1	2	Stroll (1010101010)	
2	3	Colapinto (1011001100)	
3	3	Alonso (1010001000)	Verstappen (1110101000)
4	3	Hamilton (1001001011)	
5	3	Leclerc (1010100111)	Sainz (1010001111)

Al dar de baja a Alonso (1010001000), el bloque 3 queda vacío. Este bloque tenía nivel de dispersión local igual al global (3). Se identifica su bloque hermano: aquel que comparte el mismo sufijo salvo en el bit más significativo utilizado para el direccionamiento, en este caso

el bloque 2 (sufijo 100). Como el bloque hermano también tiene un nivel de dispersión local de 3 y contiene claves válidas, se libera el bloque 3 y su rango es absorbido por el bloque hermano. Luego, se reduce en uno el nivel de dispersión local del bloque resultante y se actualizan las entradas de la tabla que apuntaban al bloque eliminado para que redirijan al bloque que permanece.

Tabla de dispersión Bits de dispersión: 3	
Sufijo	#Bloque
(000)	2
(001)	0
(010)	1
(011)	4
(100)	2
(101)	0
(110)	1
(111)	5

Archivo de datos			
#Bloque	Bits	Clave R1	Clave R2
0	2	Russell (1010001001)	
1	2	Stroll (1010101010)	
2	2	Colapinto (1011001100)	
3	3	Alonso (1010001000)	
4	3	Hamilton (1001001011)	
5	3	Leclerc (1010100111)	Sainz (1010001111)